

Resolving Ambiguities in Natural Language Software Requirements: A Comprehensive Survey

Unnati S. Shah

Department of Computer Engineering

C. K. Pithawalla College of Engineering and Technology

Near Malvan Mandir, Dumas Road, Surat-395007

+919033473552

unnati.shah25@gmail.com

DOI: 10.1145/2815021.2815032

ABSTRACT

<http://doi.acm.org/10.1145/2815021.2815032>

Requirements Engineering is one of the most vital activities in the entire Software Development Life Cycle. The success of the software is largely dependent on how well the users' requirements have been understood and converted into appropriate functionalities in the software. Typically, the users convey their requirements in natural language statements that initially appear easy to state. However, being stated in natural language, the statement of requirements often tends to suffer from misinterpretations and imprecise inferences. As a result, the requirements specified thus, may lead to ambiguities in the software specifications. One can indeed find numerous approaches that deal with ensuring precise requirement specifications. Naturally, an obvious approach to deal with ambiguities in natural language software specifications is to eliminate ambiguities altogether i.e. to use formal specifications. However, the formal methods have been observed to be cost-effective largely for the development of mission-critical software. Due to the technical sophistication required, these are yet to be accepted in the mainstream. Hence, the other alternative is to let the ambiguities exist in the natural language requirements but deal with the same using proven techniques viz. using approaches based on machine learning, knowledge and ontology to resolve them. One can indeed find numerous automated and semi-automated tools to resolve specific types of natural language software requirement ambiguities.

However, to the best of our knowledge there is no published literature that attempts to compare and contrast the prevalent approaches to deal with ambiguities in natural language software requirements. Hence, in this paper, we attempt to survey and analyze the prevalent approaches that attempt to resolve ambiguities in natural language software requirements. We focus on presenting a state-of-the-art survey of the currently available tools for ambiguity resolution. The objective of this paper is to disseminate, dissect and analyze the research work published in the area, identify metrics for a comparative evaluation and eventually do the same. At the end, we identify open research issues with an aim to spark new interests and developments in this field.

Categories and Subject Descriptors

D.2.1 [Software Engineering]: Requirements/Specifications – Elicitation methods (e.g., rapid prototyping, interviews, JAD), Languages, Methodologies (e.g., object-oriented, structured), Tools.

General Terms

Documentation, Performance, Languages, Theory, Verification.

Keywords

Requirements Engineering, Ambiguity, Natural Language Processing.

1. INTRODUCTION

Requirements Engineering (RE) is the activity that involves the functions associated with the extraction, modeling, analysis, verification and specification of the user's requirements [1]. The RE activity often starts with the vaguely defined requirements [2] and results eventually in to a Software Requirements Specification (SRS) document. The SRS is a part of the contract and it must describe the user and the system requirements clearly, accurately and unambiguously. An SRS that has

Devesh C. Jinwala

Department of Computer Engineering

S. V. National Institute of Technology

Ichchhanath, Surat-395007

+919426611777

dcjinwala@gmail.com

inconspicuous, incomplete, unmanaged, unspecified, inaccurate or ambiguous requirement definition may eventually lead to cost and time overruns [3, 4, 5]. A major open research problem in RE is resolving ambiguity. An ambiguity is defined as "a statement having more than one meaning". An ambiguity can be either *lexical*, *syntactic*, *semantic*, *pragmatic*, *vagueness*, *generality* and *language error* ambiguity [6]. One of the panaceas to preventing ambiguities in requirements specifications is to ensure that the requirements are specified in a formal language with mathematical notations that lend precision [7, 8]. However, the use of formal language specifications in common domains is still lacking due to the need for the additional skill and expertise [9]. Hence, despite the fact that the requirements specified in natural language tend to erroneous interpretations due to the inherent ambiguities associated, the requirements are most often stated in natural language. Therefore, it is necessary to investigate the approaches that deal with resolving the ambiguities from the user requirement specifications.

Manually resolving ambiguity from software requirements is a tedious, time-consuming, error-prone, and therefore expensive process [6]. Moreover, as systems become able to handle high quality industrial applications, their requirements become massive, further increasing scope of ambiguities creeping in [10]. Thus, an automated and semi-automated approach to resolve ambiguities from the requirements statement is desirable.

There exist diverse approaches, starting from manual glossaries approach to automatic ontology based approach to reduce ambiguity from the SRS. In addition, there are a number of different tools viz. WSD [11], QuaARS [12], ARM [13], RESI [14], SREE [15, 16], NAI [17, 18], SR-Elicitor [19], and NL2OCL [20] developed to detect and resolve ambiguities. However, as per our observations, there is a lack of any attempt to discuss a comparative evaluation of the approaches for dealing with ambiguities as well as the tools proposed for the purpose. Hence, in this paper, we attempt to present an insight into how current ambiguities resolving tools work, the approach followed by each tool, the types of ambiguities resolved and the features they support. We use the performance measures such as *Recall*, *Precision* and *F-measure* to comparatively analyze the performance of the ambiguity resolving tools.

2. BACKGROUND

Success of any software is based on how well it reflects the specifications document. The specification document must in turn mirror the user's actual requirements. Hence, the major issue in ensuring the success of software lies in extracting the correct requirements from a user and then in specifying them using an appropriate technique for ensuring unambiguous design. There are two ways of specifying software requirements viz. (1) specifying requirements using formal specification approaches (2) doing so, using informal specifications [21]. From our literature survey, we observe that the use of informal natural language or semi-formal approaches (UML) to specify requirements is generally preferred. This is so because if it stated formally, the stakeholders find it difficult to take part in further discussions about the requirements [22]. Semi-formal (UML) and

formal languages (viz. OCL, Z) [7, 8] are advantageous in that they have a well-defined semantics that reduce the degree of ambiguities in SRS. However, using semi-formal languages in inherently entails issues viz. *all the stakeholders have to write requirements in the same language; they have to learn syntaxes and develop skills to do so* and so on. Hence, the requirements are initially written in natural language and then transformed into a formal language [9]. On the other hand, informal representation causes misinterpretation due to ambiguous nature of natural language.

Thus, there can be two approaches to deal with the ambiguities viz. either ambiguity prevention or ambiguity detection and resolution. In the first approach, the aim is to prevent ambiguities from appearing in the requirements specifications altogether. This approach warrants that the requirements have to be stated formally i.e. using precise statements that have a unique meaning. The typical approaches to do so use controlled languages, context free grammar, style guides/sentence pattern. In the second approach, the ambiguities are not prevented outrightly, but are allowed to remain in the initial requirements specifications document; however later appropriate techniques viz. machine learning, knowledge based and ontology are used to detect and resolve them.

2.1 Ambiguity

Ambiguous requirements lead to confusion, wasted effort and rework. An ambiguity has two sources: missing information and communication errors. Errors are categorized in two ways. The first one is *author-independent* errors – ones, that can be solved without domain knowledge e.g. “grammatical errors.” The other is *author-dependent* errors – ones that need domain knowledge to resolve e.g. “*lack of detail*” to correct the error [6, 9].

There are two broad categories of ambiguities viz. linguistic ambiguities and software engineering ambiguities. The Language ambiguities are ambiguities that can be spotted by any reader who has an ear for language. Ambiguity Handbook [6] presents different types of Linguistic ambiguities. In contrast, software engineering ambiguities are ambiguities that depend on the domain involved and can be spotted only by readers that have sufficient domain knowledge.

Table 1. Types of Ambiguity

	Type of Ambiguity	Subtype
Language Ambiguity	Lexical Ambiguity	Homonym Ambiguity Polysemy Ambiguity
	Syntactic Ambiguity	Analytical Ambiguity, Attachment Ambiguity, Coordination Ambiguity, Elliptical Ambiguity
	Semantic Ambiguity	Scope Ambiguity
	Pragmatic Ambiguity	Referential Ambiguity, Deictic Ambiguity
	Vagueness, Language Error, Generality	
RE-Specific Ambiguity	Conceptual Translational Ambiguity	
	Requirements Document Ambiguity	
	Application Domain Ambiguity	
	System Domain Ambiguity Development Domain Ambiguity	

The Software engineering ambiguity accounts for the majority of ambiguities, while pure language ambiguities play less significant role.

Authors in [9] suggest various types of RE-specific ambiguities. In Table1. we summarize Language ambiguity and RE-specific ambiguity. The detailed description with relevant examples is given in [6, 9] wherein [6] focuses on language ambiguity and [9] focuses on RE-Specific ambiguity.

3. APPROACHES TO DETECT AND RESOLVE AMBIGUITY

As of now evident, since the requirements specified in a natural language tend to be ambiguous, it is necessary that such requirement specification document is first per-processed. The purpose of such per-processing is to identify ambiguous sentences in an SRS and to help the user to understand why it is ambiguous, so it can be removed, and thus to improve the SRS. For the purpose, the Natural Language Processing (NLP) techniques are used. The possible usage of NLP techniques are: extract requirements from the document, tag the requirements sentence, find duplicate requirements, do the machine translation and extract the ambiguous requirements. Due to the complexity of natural language requirements, the authors in [23] state that “NLP is not appropriate to be used as the resulting reliability of the system are doubtful”. In opposed to that, Kof and L. [24] state that “NLP usage in RE is not to understand the text, but to extract concepts contained in NL document”. Whereas authors in [9] suggest two possible ways: one is to use a constrained natural language to write specifications; it is a subset of the natural language ruled by an underlying ontology and the second is to use a linguistic-driven inspection of the requirements.

The basic NLP issues are Part-of-Speech (POS) tagging, parsing and ambiguity [25]. POS tagging marks every word of a sentence with predefined parts-of-speech (noun, verb, adjective, etc.). For example, the sentence “You have to face the problem”, marked with “You/PRP have/VBP to/TO face/VB the/DT problem/NN” here NN means “noun”, VB a “verb”, DT a “determiner”, VBP a “verb”, and PRP a “preposition”. Taggers [26, 27] (such as Unigram, n-gram, Brill) are useful for NLP and partial parsing. The process of tagging becomes difficult when the word is ambiguous, having more than one POS. For example consider following two sentences:

- (a) I want a book.
- (b) I want to book a ticket.

In the first sentence (a) the word “book” is a “noun” and in the second sentence (b) the word “book” is a “verb”. Rule based, Statistical or Probabilistic based and Hybrid or Transformation based approaches are available to resolve tag ambiguity [28]. **The rule based approach** is extremely laborious because it requires keeping the rules up to date that cover all cases including exception cases. However, it is not sufficient to get the solution only using the rule based approach, as the context knowledge is also required. **The statistical based approach** is based on the training data set. Such an approach requires a voluminous data and hence an inherent issue is the access to the data. **The hybrid approach** combines the features of both rule based approach and statistical approach [28]. In [15, 16] authors put their effort to resolve ambiguities based on guideline pattern provided in [6] except vagueness, referential ambiguity and other complex structures. They use statistical based Tree Tagger to tag the requirements. Parsing a text (using Stanford [30], Dowser [29, 31]) is helpful to detect or disambiguate ambiguous words by looking at the syntax – however such techniques are useful for lexical and syntactic ambiguities. Stanford Parser builds on a model that relies on the general text viz. newspaper and scientific text. The Stanford parser gives acceptable accuracy if the input text is similar to the text used to build the model. It gives a wrong answer for the ambiguous words e.g. watch, book, cut, etc. Furthermore, it gives dependencies - a grammatical relation between a head and a dependent relative in a sentence [30]. In [20] authors identify cases where the Stanford parser generates a correct parse tree, but the typed dependencies go wrong because of syntactic ambiguity. They focus on attachment/concord ambiguity and homonym

ambiguity. The authors suggest following steps to resolve the ambiguities:

1. Input English sentences and its UML class model.
2. Parse the sentences using Stanford parser.
3. Perform syntactic analysis.

They perform syntactic analysis by comparing dependencies with UML class model (viz. classes, methods, attributes, associations, etc.). It leads to achieve Stanford parser accuracy from 84.1% to 92.8% for attachment ambiguity and from 97.0% to 99.0% for homonym ambiguity [20]. However, there are possibilities of ambiguities in an input UML class model that ultimately introduce ambiguities in the results.

3.1 Checklist Based Inspection Approach

Checklist based inspection approach identifies ambiguities by giving requirements to stakeholders from whom an interpretation is sought, and compare the interpretation. Diverge interpretation of the requirements stated as ambiguous [32]. Fagan et al. [33] comprise different categories of checklist items (viz. desirable checklist and undesirable checklist) and a survey of 117 checklists from 24 sources. They present an efficacy of the approach in the software development process. However, users are not aware of ambiguities. Hence simply asking them for ambiguities is not much useful. In [9] authors argue that checklist based inspection approach is not efficient to find the ambiguities in requirements that are far away in SRS. They apply scenario based reading approach with a checklist to enhance the ambiguity detection rate. The scenario based reading provides operational scenarios while analyzing the requirements to find errors. Their work focuses on RE-specific ambiguity in industrial SRS.

3.2 Style Guides

Style guides are used to avoid syntactic and semantic ambiguities by allowing input requirements in a specific manner (e.g. use active voice instead of passive voice, include braces, insert comma etc.). Sentence pattern is an alternative approach to avoid ambiguities where requirements are rewritten with minor modifications to match the predefined patterns. There is an obvious need for tool that identifies the ambiguous sentences from the natural language SRS. The tool suggests a way to remove the ambiguities for each ambiguous sentence and improve the quality of SRS [34]. Sri Fatimah Tjong et al. [15, 16] propose a tool SREE (Synthesized Requirements Engineering Environment) that detects an instance of potential ambiguities in the software requirements and report it to the user. User rewrites the requirements if the instance is truly ambiguous. The authors suggest guiding rules that detect ambiguities due to incompleteness, vagueness, under-reference, quantifier, pronoun and plural. They use a lexical analyzer for scanning and context-free grammar for parsing. The tool explores instances for specific words in SREE's database. Hence SREE provide 100% recalls for potential ambiguities in SREE's scope. However, the authors find cases where the ambiguities detected, but no suggestions are available to rewrite the requirements. Recall, precision and F-measure are three basic parameters to measure the performance of ambiguity detecting and resolving tools [37, 45].

$$\text{Recall} = I_{\text{correct}} / I_{\text{total}}$$

$$\text{Precision} = I_{\text{correct}} / (I_{\text{correct}} + I_{\text{incorrect}}) \quad (\text{I})$$

$$\text{F-measure} = 2 * \text{precision} * \text{Recall} / (\text{precision} + \text{Recall})$$

Independent parameter recall and precision are not worthy to decide the performance of the tool as it is obvious to achieve 100% recall, but then precision suffers, and vice versa. For example, all words marked as ambiguous, gives a recall of 100%. Conversely, single word in 100 words is truly ambiguous gives a precision of 1%. High ambiguity detection rate requires high recall and low error rate requires high precision. Recall and precision should be examined together using

F-measure parameter [35, 36]. Methods for calculating recall, precision and F-measure are as mentioned in Eq. (I). The authors present a comparison of their tool SREE with ARM [13] and TIGER Pro [37]. SREE provides guiding rules that cover ambiguity indicators and rescue the corrected version of requirements after rewriting. It accepts input requirements in the form of text file (.txt) or document file (.doc). However, SREE does not support graphical analysis that is provided by TigerPro. The tool SREE is not capable to deal with vague ambiguity, plural noun ambiguity and complex coordination ambiguity.

3.3 Controlled Language

A Controlled Language is used to reduce the ambiguities by restricted grammar and a fixed vocabulary [6, 9]. It improves readability and provides automatic semantic analysis. Languages are formed using grammar, e.g. grammar $G = (V, T, S, PR)$, where V is a set of variables, T is a set of terminal symbols, S is a start symbol and PR is a set of production rules. The corresponding language of G is $L(G)$.

Let $V = \{S, NP, VP, N, V, Art\}$

$T = \{girl, chocolate, bike, likes, the, boy, ate\}$

$PR = \{S \rightarrow NP VP, NP \rightarrow N, NP \rightarrow Art N, VP \rightarrow V NP, N \rightarrow girl | boy | chocolate | bike, V \rightarrow ate | like, Art \rightarrow the\} \quad (\text{II})$

Following sentences can be generated using Eq. (II):

- a. *The girl likes chocolate.*
- b. *Boy ate the bike.*
- c. *The Girl ate the chocolate.*
- d. *Boy like the girl.*
- e. *Chocolate likes the bike.*

Even though sentences a, b, c, d and e are structurally correct, sentences b and e are not meaningful sentences. Hence a grammar cannot guarantee the generation of meaningful sentences. Grammars are defined by Chomsky hierarchy, as type 0, 1, 2, 3. The types 1, 2, 3 are called context-sensitive, context-free, and regular grammars respectively. The formal languages are based on the type-2 languages, as the type 0 and 1 are difficult to implement and understand [38]. Ambiguous grammars have more than one parse-tree. Bas Basten and Tijs Van Der Storm present an ambiguity detection tool AMBIDEXTER based on context free grammar [39]. The tool combines exhaustive searching and approximate searching. Exhaustive searching generates a language of grammar of all sentences and check for multiple parse trees. Approximate searching uses the grammar generated in exhaustive searching and produces comprehensive ambiguity reports effectively. Attempto Controlled English (ACE) [40] is a subset of English with a restricted grammar and a domain-specific vocabulary that unambiguously translate sentences into first-order logic. Though it is a formal language, it looks natural language. Hence effortlessly interpreted by human and machine. ACE replaces ambiguous sentence constructs by unambiguous sentence constructs [40]. Controlled Languages might be impractical if they are excessively restricted. This problem is addressed in [41], where they first construct SRS using a controlled language and then tailor it to the sub language of SRS. The approach detects and resolves typical syntactic and semantic ambiguities from SRS. However, detection of referential ambiguity, provision for lexical incompleteness and reporting missing words need to be researched. They enhance Alvey Natural Language Toolkit (ANLT) with features viz. parse selection and error investigation. ANLT was initially contained facilities for viewing sentences and its parse trees. Daniel M. et al. [42] present a semi-automated approach that reduces the number of ambiguities, inconsistencies, and under specifications in a natural language SRS through automation. The authors combine the strength of automation and human reasoning. They present a three step semi-automated approach:

1. Parse a natural language SRS according to a constraining grammar.
2. Elicit object oriented terms to create an object-oriented model.
3. Detect ambiguities and inconsistencies.

They use static dowser parser to construct the grammar. Thus the tool is limited to deal with static relationships among the concepts mentioned in the parsed natural language SRS and cannot deal with dynamic modal verbs and negations e.g. ‘not’. They achieved a reasonable error detection rate 78.8% (for compound-noun) and 93.9% (for single noun). To achieve efficient results, static and dynamic approaches can be combined. Approaches that have been discussed so far need lots of manual process. Automated approaches can be clarified with the following example: “*I saw the balloon with the telescope.*” The correct interpretation of the sentence is “*one should see the balloon using the instrument telescope and not that balloon is having telescope*”. Context knowledge required to disambiguate the sentence.

3.4 Knowledge Based Approach

A single word may have diverged meaning in input text with a range of ambiguities. To extract the most appropriate meaning is a challenging task. To extract and manipulate the meaning of the text, NLP system must have an extensive knowledge about the world and the domain of discourse. Knowledge based approach applies this knowledge to resolve ambiguities. The knowledge based system can be viewed as a search system that uses different types of knowledge with a view to constrain the search space. It offers optimal, acceptable and efficient search results. The Knowledge based NLP systems are domain specific. To make it domain independent, language specific (i.e. lexical) knowledge and domain/world knowledge are separated, called ontology [44]. The word ontology has recently gained a good popularity within the knowledge engineering community. Ontology based approach [45, 19, 46, 47, 48, 49, 50, 51] provides a strong and unambiguous communication mechanism, portability, interoperability and reusability.

Sven Körner and Torben Brumm [14] develop a semi automated natural language specification improver tool RESI (Requirements Engineering Specification Improver) based on knowledge of ontologies. The authors focus on issues viz. ambiguities, normalization and incompleteness. They provide a user friendly dialog based tool to identify errors. The tool provides an auto generated suggestions and inquiries to resolve the errors. However, an input text specification is limited to GrGen graphic format. Recently the authors provide nlrp BENCH [36] a benchmark for natural language requirements processing. They provide an open platform that contains over 50 requirements specification for the quality check of the RE tool. The tool elicits object oriented terms, identifies errors and provides suggestions using ontologies [6, 36]. Compare to manual process their tool provide 30-80% higher error detection rate. Nowadays, collaboration across several organizations essentially requires clarity and unambiguity of knowledge transfer. Individual requirement engineers have their own contextual boundaries of interpreting a unit or set of information. Since all requirement engineers involve in a collaborative activity, requires an equal understanding of knowledge [52]. Object Management Group (OMG) has presented specification of ‘Semantics of Business Vocabulary and business Rules’ (SBVR) for the field of business [53]. The SBVR has mathematical foundation that results in accurate, consistent and machine processable software models. Transfer natural language to object constrained language automatically using SBVR is addressed in [54]. The whole procedure is divided into following steps:

1. Parse natural language software requirement text.
2. Extract SBVR vocabulary (e.g. object type, fact type characteristics, quantification).
3. Generate SBVR rules by extracting structural requirements (using auxiliary verbs e.g. ‘can’, ‘may’), behavioral requirements (using auxiliary verbs e.g. ‘should’, ‘must’).
4. Apply semantic formulation (viz. logical, quantification or model).

Knowledge and Ontology based approach propose in [55] called Context Knowledge & Concepts Ontology (CKCO) for resolving lexical-semantic ambiguity in natural language. The context knowledge identifies the correct sense of the word in the text using WordNet with domain labels. The concept knowledge identifies the concept and relates the concepts. The work focuses on natural language questions. The authors inspect the approach for simple sentence construction with low error rate and achieve precision 83.4%. In [49] authors present a module that takes the requirement specification document, resolves the ambiguities using knowledge based approach, develops the ontology for the system and derive the non-functional requirements by comparing the use-case description with the domain model.

3.5 Heuristics Based Approach

The approaches based on machine learning [56, 57] use a corpus as an example, deducing further knowledge. Such approaches usually guarantee a solution with high probability. In addition, employing heuristics [18, 17], in these approaches, one can realize deterministic solution with defined bound. The machine learning approach proposed in [25] to address prepositional phrase disambiguation. Accuracy of a machine learning based system depends on the size of the corpus on which it is trained. The work proposed in [17] focuses to detect nocuous coordination ambiguities that carry a high risk of misunderstanding among readers. The authors present an automatic tool NAI (Nocuous Ambiguity Identification) that identifies coordination ambiguity especially noun compound and verb compound from the SRS. To deal with the ambiguity the users are helped with heuristics based on corpus based statistical and human judgment. They prove the effectiveness of machine learning algorithm. To predict nocuity effectively authors [17] added collocation frequency heuristics and semantic similarity heuristics compare to [56]. The tool achieve 70% precision and 100% recall for coordination ambiguity and 82.4% precision and 74.2% recall for anaphora ambiguity with medium user interaction. To enhance the accuracy of the tool heuristics need to be developed that explore aspects of ambiguity further viz. the sentence contain chained coordination ambiguity (e.g. “A and B and C”), the sentence starts with coordination itself (e.g. “And B”), the sentence contain the combination of “and/or” (e.g. “A and B or C”). The authors present an analysis of semantic ambiguities viz. anaphora using heuristics [18].

So far we have discussed varieties of disambiguation approaches and recent tools. Table 2 shows the comparative analysis of discussed tools based on parameters viz. approach followed by the tool, technologies used to perform tagging and parsing, per-processing of requirements supported or not, concern ambiguity, level of user interaction required (viz. medium, low, high), elicit object oriented terms or not, and relevant information as remarks. Resolve all ambiguities from the requirements specification are a tedious task and to make it completely automated is not feasible. We need human interaction to resolve complex ambiguities that depend on the domain knowledge. We observe that each tool is intended to deal with a specific type of ambiguity. There exists no tool that deals with all kinds of ambiguities except the good try given by [15, 16] using rule based approach with significant limitation. Controlled language is good to identify or to avoid the ambiguities in SRS with the limitation that input should be written in the constraint language and is dealt with lexical and syntactic ambiguities. Moreover, tools that use knowledge based, machine learning and ontology approaches are able to give precise results by identifying semantic ambiguities from requirements specification. Furthermore, tools [9, 15, 16, 19, 55] require normalization to obtain accurate results. In addition, tools [9, 20, 22] identify the object oriented terms that are useful for semi-formal representation viz. UML, OCL with low, medium or high user interaction.

Table 2. Comparative Analysis of Ambiguities Resolving Tools

Feature Support	Approach	Technologies used	Pre-processing	Concerned Ambiguity	User interaction	Elicit OOT	Remark
OOV of NLRs (Automatic) (2006)	Knowledge based to ontology	Brill tagger, GATE tool	Yes	Pronoun anaphora	Medium	Yes	Nonfunctional requirements elicitation. Ontology generation.
RA in RS via OOM (Semi-automatic) (2008)	Controlled language	Dowser parser	No	Semantic	Medium	Yes	Cannot deal with modal verbs and negations. Recall 78.8% (Compound-noun) Recall 93.9% (Single noun)
SREE (Semi-automatic) (2008)	Rule based, Style guide	WordNet, POS tagger	No	Identify Plural, Coordination, Pronoun, Quantifier, Vague	Low	No	Report summary of ambiguous and incomplete requirements statements. Recall 100% (w.r.t SREE's scope)
RESI (Semi-automatic) (2009)	Knowledge based to ontology	Stanford parser, Cyc, ConceptNet, WordNet	Yes	Avoid Lexical, Scope, Language Error	High	Yes	Input must be in the graph GrGen format.
NAI (Automatic) (2010,2011)	Machine learning/heuristics based	LogitBoost, Named entity recognition	Yes	Noun and Verb compound coordination, Anaphora ambiguity	Medium	No	Establish the degree of nocuity that the system should tolerate. Precision 70% and Recall 100% (Coordination) Precision 82.4% and Recall 74.2% (Anaphora)
SR-Elicitor (Automatic) (2011)	Controlled language	SBVR, POS tagger	No	Lexical, Syntactic, Scope-Quantifier	Low	Yes	SBVR rule generation. Recall 80.12% and Precision 85.76%
NL2OCL (Automatic) (2012)	Knowledge based to ontology	SBVR, Stanford parser	No	Attachment , Homonymy	Low	Yes	A UML class model is required as an input. Recall 92.85% Precision 92.85% (Attachment) Accuracy 99.0% (Homonymy)
CKCO (Automatic) (2012)	Knowledge based to ontology	WordNet, WSD	No	Lexical Polysemy (ambiguity of nouns)	Low	No	Resolve ambiguity posed to a Question Answering (QA) system Precision 83.4%

4. Conclusions

The correct and precise software requirements specification is required for the success of the software. Although the formal methods can be used to generate precise software specifications, the use of formal methods has not yet percolated to the mainstream. Hence, the alternate recourse that is often pursued to allow ambiguities prevails in the requirements document and subsequently, using appropriate techniques to identify and eliminate them before generating the software specifications. Motivated by the importance of resolving ambiguities, in this paper, we survey the state-of-the-art in approaches for resolving ambiguities in natural language requirements. We review and discuss different approaches and tools for resolving ambiguities in the software requirements specification. We observe that the tools are broadly classified as automated and semi-automated. The readability and understandability of the requirements increases by applying domain specific language, restricted syntax/grammar and sentence patterns.

However, it requires lot of human process, significant expertise and complex to apply in every environment. On the other hand, tools that use knowledge based to ontology and machine learning approaches are efficient and give accurate results by identifying semantic ambiguities. However, majority tools use natural language processing tools viz. Stanford parser, dowser parser that is still under improvements. Accuracy of the disambiguation tools depends on the parser they use. If the parser does not identify the right tokens and their dependencies, then the whole process becomes inadequate, ultimately leads to a waste of efforts. Furthermore, software requirements are not fixed in nature, the unpredictable change in the requirements may introduce new ambiguities. Thus, tools work efficiently with the predefined requirements, but get complex with the changing nature of requirements. As we observe, using a tool to resolve the ambiguities in requirements, though fruitful, does not eliminate the ambiguities automatically and completely. Furthermore, the applicability of such tools cannot be generic and is limited. As per our observations, this

appears to be the most promising approach to the challenge of formal representation of knowledge.

5. REFERENCES

- [1] Sommerville, I. and Sawyer, P. 1997. *Requirements Engineering A good practice guide*. Chichester: John Wiley & Sons Ltd.
- [2] Nuseibeh, B., & Easterbrook, S. 2000, May. Requirements engineering: a roadmap. In *Proceedings of the Conference on the Future of Software Engineering* (pp. 35-46).
- [3] Belev, G. C. 1989, January. Guidelines for specification development. In *Reliability and Maintainability Symposium, 1989. Proceedings., Annual* (pp. 15-21). IEEE.
- [4] Christel, M. G., & Kang, K. C. 1992. Issues in requirements elicitation (No. CMU/SEI-92-TR-12). CARNEGIE-MELLON UNIV PITTSBURGH PA SOFTWARE ENGINEERING INST.
- [5] Donald G. Firesmith. 2007. Common Requirements Problems, Their Negative Consequences, and Industry Best Practices to Help Solve Them. In *Journal of Object Technology*, vol. 6, no. 1, January-February 2007, pp. 17-33
- [6] Berry, D. M., Kamsties, E., & Krieger, M. M. 2003. From contract drafting to software specification: Linguistic sources of ambiguity. Technical Report, University of Waterloo, Waterloo, Canada. Available at. <http://se.uwaterloo.ca/~dberry/handbook/ambiguityHandbook.pdf>
- [7] Hutchison, D., Araki, K., Kanade, T., Kittler, J., Kleinberg, J. M., Liu, S. and Weikum, G. 2008. Formal Methods and Software Engineering: 10th International Conference on Formal Engineering Methods, ICFEM 2008, Kitakyushu-City, Japan, October 27-31, 2008. *Proceedings*. Berlin, Heidelberg: Springer Berlin Heidelberg.
- [8] Wordsworth, John B. 1992. *Software development with Z: a practical approach to formal methods in software engineering*. Addison-Wesley Longman Publishing Co., Inc., 1992.
- [9] Kamsties, E., & Peach, B. 2000, December. Taming ambiguity in natural language requirements. In *Proceedings of the Thirteenth International Conference on Software and Systems Engineering and Applications*.
- [10] Zave, Pamela. 1997. Classification of research efforts in requirements engineering. *ACM Computing Surveys (CSUR)* 29.4 (1997): 315-321.
- [11] Nancy Ide and Jean Véronis. 1998. Introduction to the special issue on word sense disambiguation: The state of the art. *Computational Linguistics - Special issue on word sense disambiguation*, Volume 24 Issue 1,2-40.
- [12] Fabbrini, F., M. Fusani, S. Gnesi, and G. Lami. 2001. The Linguistic Approach to the Natural Language Requirements Quality: Benefit of the use of an Automatic Tool. SEW'01 proceeding of the 26th annual NASA Goddard Software Engineering Workshop, IEEE Computer Society Washington, DC, USA, 97.
- [13] Willis, Alistair, Francis Chantree, and Anne De Roeck. 2008. Automatic Identification of Nocuous Ambiguity. *Research on Language & Computation*, 6 (3-4), 1-23.
- [14] Sven Körner and Torben Brumm. 2009. RESI-A natural language specification improver. *IEEE International Conference on Semantic Computing (ICSC)*.
- [15] Sri Fatimah Tjong. 2008. Avoiding ambiguity in requirements specifications. Thesis submitted to the University of Nottingham for the degree of Doctor of Philosophy.
- [16] Tjong, Sri Fatimah, and Daniel M. Berry. 2013. The Design of SREE—A Prototype Potential Ambiguity Finder for Requirements Specifications and Lessons Learned. *Requirements Engineering: Foundation for Software Quality*. Springer Berlin Heidelberg, 2013. 80-95.
- [17] Hui Yang, Alistair Willis, Anne De Roeck, Bashar Nuseibeh. 2010. Automatic Detection of Nocuous Coordination Ambiguities in Natural Language Requirements. *Proceedings of the IEEE/ACM international conference on Automated software engineering*, 53-62. ISBN: 978-1-4503-0116-9. DOI=10.1145/1858996.1859007.
- [18] Hui Yang, Anne de Roeck, Vincenzo Gervasi, Alistair Willis Bashar Nuseibeh. 2011. Analyzing anaphoric ambiguity in natural language requirements. *Requirements Engineering - Special Issue on Best Papers of RE'10: Requirements Engineering in a Multifaceted World*, Volume 16 Issue 3, 163-189. DOI=10.1007/s00766-011-0119-y.
- [19] Basili, Victor R., Scott Green, Oliver Laitenberger, Filippo Lanubile, Forrest Shull, Sivert Sorumgard. 1995. *The Empirical Investigation of Perspective-Based Reading*. Technical report the empirical investigation of perspective based reading.
- [20] Imran SarwarBajwa. 2012. Resolving Syntactic Ambiguities in Natural Language Specification of Constraints. *CICLing'12 Proceedings of the 13th international conference on Computational Linguistics and Intelligent Text Processing*, Volume 1, 178-187.
- [21] Abderrahman Matoussi and RégineLaleau. 2008. A Survey of Non-Functional Requirements. In *Software Development Process Technical report TR-LACL-2008-7*, LACL (Laboratory of Algorithms, Complexity and Logic), University of Paris-Est (Paris 12).
- [22] Heitmeyer, Constance L., Ralph D. Jeffords, and Bruce G. Labaw., 1996. Automated Consistency Checking of Requirements Specifications. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, ACM vol. 5, no. 3, 231-261. DOI=>10.1145/234426.234431.
- [23] Ryan and K. 1993. The role of natural language in requirements engineering. *Proceedings of the IEEE Int. Symposium on Requirements Engineering*. San Diego, CA, 240-242.
- [24] Kof and L. 2004. Natural Language Processing for Requirement Engineering: Applicability to large Requirements Documents. Available at. http://www.dsl.uow.edu.au/~jp989/Scalability_WITSE04.pdf
- [25] Eric Brill. 1995. Transformation-Based Error-Driven Learning and Natural Language Processing: A Case Study in Part-of-Speech Tagging. *Journal computational linguistics volume 21 issue 4* December, 543-565.
- [26] Robin. 2009. POS-Tagging. Available at <http://language.worldofcomputing.net/pos-tagging>, Dec.4th 2009.
- [27] Tagger. Available at. <http://nltk.googlecode.com/svn/trunk/doc/howto/tag.html>
- [28] Hajič, J., Votrubec, J., Krbec, P., & Květoň, P. 2007, June.. The best of two worlds: Cooperation of statistical and rule-based taggers for Czech. In *Proceedings of the Workshop on Balto-Slavonic Natural Language Processing: Information Extraction and Enabling Technologies* (pp. 67-74). Association for Computational Linguistics.
- [29] Klein, Dan, and Christopher D. Manning. 2003. Accurate unlexicalized parsing. *Proc. of the 41st Annual Meeting on Association for Computational Linguistics-Volume 1*, 2003, pp. 423-430
- [30] Cer, Daniel M., Marie-Catherine De Marneffe, Daniel Jurafsky, and Christopher D. Manning. 2010. Parsing to Stanford Dependencies: Trade-offs between Speed and Accuracy. *LREC*, 2010
- [31] Popescu, D., Rugaber, S., Medvidovic, N., & Berry, D. M. 2008. Improving the quality of requirements specifications via automatically created object-oriented models. *Innovations for Requirements Engineering*, 71.
- [32] Kamsties, Erik, Daniel M. Berry, and Barbara Paech. 2001. Detecting ambiguities in requirements documents using inspections. *Proceedings of the First Workshop on Inspection in Software Engineering (WISE'01)*, 68-80.
- [33] Fagan, Michael. 2002. Reviews and inspections. *Software Pioneers—Contributions to Software Engineering*, 562-573.
- [34] Kiyavitskaya, Nadzeya, Nicola Zeni, Luisa Mich, and Daniel M. Berry. 2008. Requirements for tools for ambiguity identification

- and measurement in natural language requirements specifications, Requirements Engineering 13, no. 3, 207-239.
- [35] McSherry, Frank, and Marc Najork. 2008. Computing information retrieval performance measures efficiently in the presence of tied scores. Advances in information retrieval. Springer Berlin Heidelberg, 2008. 414-421.
 - [36] Tichy, W. F., Landhäuser, M., Körner, S. J., & Fasanengarten, A. nlrpBENCH: A Benchmark for Natural Language Requirements Processing. Manuscript submitted for publication.
 - [37] TigerPro.
<http://www.therightrequirement.com/TigerPro/TigerPro.html>
 - [38] K.R. Chowdhary. 2012. Natural Language Processing. M.B.M. Engineering College, Jodhpur, India, lecture notes 2012.
 - [39] Bas Basten and Tijs van der Storm. 2010. AMBIDEXTER: Practical Ambiguity Detection Tool Demonstration. Source Code Analysis and Manipulation (SCAM), 2010 10th IEEE Working Conference, 101-102.
 - [40] Fuchs, Norbert E., Kaarel Kaljurand, and Tobias Kuhn. 2008. Attempto Controlled English for Knowledge Representation. In Reasoning Web, Springer-Verlag Berlin Heidelberg, LNCS 5224, pp. 104-124.
 - [41] Miles Osborne and MacNisht. 1996. Processing Natural Language Software Requirement Specifications. Proceedings of ICRE, IEEE, 229 - 236.
 - [42] Popescu, D., Rugaber, S., Medvidovic, N., & Berry, D. M. 2008. Reducing ambiguities in requirements specifications via automatically created object-oriented models. In *Innovations for Requirement Analysis. From Stakeholders' Needs to Formal Designs* (pp. 103-124). Springer Berlin Heidelberg.
 - [43] Grishman, Ralph. 1997. Information extraction: Techniques and challenges. Information extraction a multidisciplinary approach to an emerging information technology. Springer Berlin Heidelberg, 1997. 10-27.
 - [44] Kavi Mahesh and Sergei Nirenburg. 1996. Knowledge-based Systems for Natural Language Processing. Chapter Simon Fraser University - Athabasca University, Canada, 1996
 - [45] Thelin, T., Runeson, P., & Wohlin, C. 2003. An experimental comparison of usage-based and checklist-based reading. Software Engineering, IEEE Transactions on, 29(8), 687-704.
 - [46] Christina Unger and Philipp Cimiano, 2011. Representing and resolving ambiguities in ontology-based question answering. TIWTE'11 proceedings of the TextInfer workshop on Textual Entailment, Springer, 40-49.
 - [47] Inah Omoronyia. 2010. A Domain Ontology Building Process for Guiding Requirements Elicitation. In: R. Wieringa and A. Persson (Eds.): REFSQ 2010, LNCS 6182, Springer-Verlag Berlin Heidelberg, pp. 188-202.
 - [48] Hassell, Joseph, Boanerges Aleman-Meza, and I. Budak Arpinar. 2006. Ontology-Driven Automatic Entity Disambiguation in Unstructured Text. In : I. Cruz et al. (Eds.): ISWC 2006, LNCS 4273, pp. 44 - 57.
 - [49] Gracia, Jorge, Vanessa Lopez, Mathieu d'Aquin, Marta Sabou, Enrico Motta. 2007. Solving semantic ambiguity to improve semantic web based ontology matching. The 2nd International Workshop on Ontology Matching, Busan, South Korea,
 - [50] Anandha Mala and Uma. 2006. Object Oriented Visualization of Natural Language Requirement Specification and NFR Preference Elicitation, In : IJCSNS International Journal of Computer Science and Network Security, VOL.6 No.8, 91-100.
 - [51] Barrett and Beum-Seuk Lee. 2002. Two-level Grammar as an Object-Oriented Requirements Specification Language. Proceedings of the 35th Annual Hawaii International Conference on System Sciences (HICSS'02) -Volume 9 - Volume 9, 280, ISBN:0-7695-1435-9.
 - [52] Norbert Jastroch and Thomas Marlowe. 2010. Knowledge Transfer in Collaborative Knowledge Management: A Semiotic View. Systemics, Cybernetics and Informatics, JSCI, Vol. 8, No. 6, 6-11.
 - [53] 'SBVR version 1.0. January, 2000.
<http://www.omg.org/spec/SBVR/1.0>
 - [54] Bajwa, Imran Sarwar, Behzad Bordbar, and Mark G. Lee. 2010. OCL constraints generation from natural language specification. In Enterprise Distributed Object Computing Conference (EDOC), 2010 14th IEEE International, 2010, pp. 204-213
 - [55] Al-Harbi, O., Jusoh, S., & Norwawi, N. 2012. Handling Ambiguity Problems of Natural Language Interface for Question Answering.
 - [56] Song, Y., Yi, E., Kim, E., Lee, G. G., & Park, S. J. 2004. POSBIOTM-NER: a machine learning approach for bio-named entity recognition. Korea, 305, 350.
 - [57] Chantree, Francis, Bashar Nuseibeh, Anne De Roeck, and Alistair Willis. 2006. Identifying Nocuous Ambiguities in Natural Language Requirements. Proceedings of 14th IEEE International Requirements Engineering Conference (RE'06), Minneapolis, USA, 59-68.